



Chapter 7 Data Modeling

Data Analytics Using Python

Dr. Ziping Liu



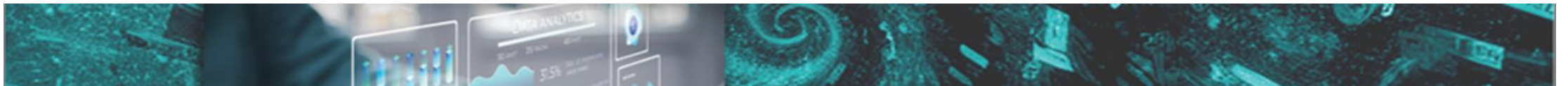
CHAPTER 7

Chapter 7 Data Modeling

<https://he.kendallhunt.com/product/data-analytics-using-python>

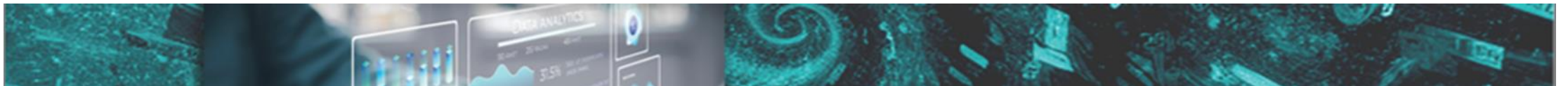
Introduction to Modeling

- develop mathematical or statistical models to represent the data distribution and make future predictions
- helps comprehend past events (descriptive analytics), understand why they occurred (diagnostic analytics), and predict future occurrences (predictive analytics)
- statistical models or machine learning models
 - Python-based libraries such as statsmodels, scikit-learn, TensorFlow, and PyTorch
 - The statsmodels package: statistical modeling, providing a broad range of traditional statistical tools
 - scikit-learn, TensorFlow, and PyTorch: machine learning modeling, offering diverse tools and algorithms for various machine learning tasks



Linear Regression

- represent a continuous and linear relationship between a dependent variable (Y) and one or more independent variables (X)
 - find the linear equation that best fits the data points of the independent and dependent variables with the linear equation minimizing the overall prediction error



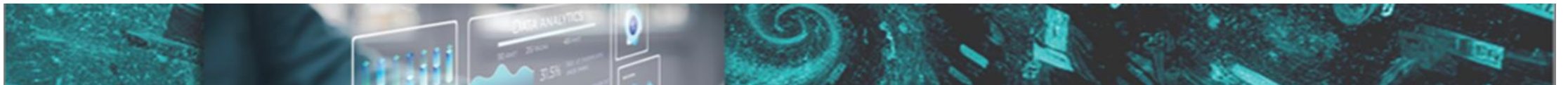
Linear Regression

- use statsmodel to perform one-way ANOVA

```
import statsmodels.api as sm
from statsmodels.formula.api import ols

# Perform one-way ANOVA
model_sm_anova = ols('Y ~ X', data=df).fit()
anova_results = sm.stats.anova_lm(model_sm_anova, typ=1)
print(anova_results)
```

- Performance measure: F-statistic and p-value



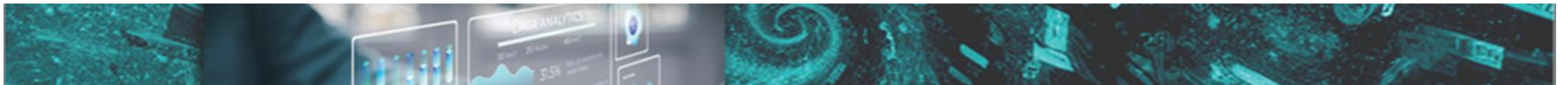
Linear Regression

- Use statsmodel's linear regression

```
from sklearn.metrics import mean_squared_error, r2_score

# Create the OLS model and fit it
model_sm_ols = sm.OLS(payments, claims_ols).fit()
Y_pred_sm = model_sm_ols.predict(claims_ols)
# Print the model summary
print(model_sm_ols.summary())
# Performance metrics
r2_sm = model_sm_ols.rsquared
mse_sm = mean_squared_error(df['Y'], Y_pred_sm)
print(f"\nstatsmodels performance - R-squared: {r2_sm:.4f}, MSE: {mse_sm:.4f}")
```

- Performance measure:
 - R-squared, adjusted R-squared values - the dependent variable
 - p-values for both the F-statistic and the coefficients - the model and the predictors statistically significant or not
 - p-values for the residual tests (Omnibus and Jarque-Bera) - normally distributed or not
 - Durbin-Watson statistic autocorrelation in the residuals



Linear Regression

- linear regression using scikit-learn library

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

model_sk = LinearRegression()
model_sk.fit(X, Y)

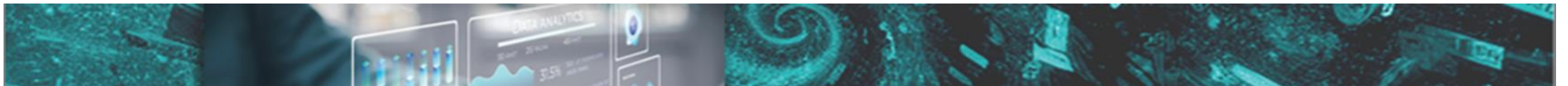
# Predicted values
df['Predicted_Y_SK'] = model_sk.predict(X)

Y_pred_sk = model_sk.predict(X)

# Performance metrics
r2_sk = r2_score(Y, df['Predicted_Y_SK'])
mse_sk = mean_squared_error(Y, df['Predicted_Y_SK'])

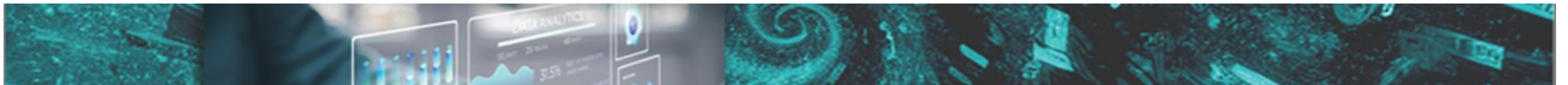
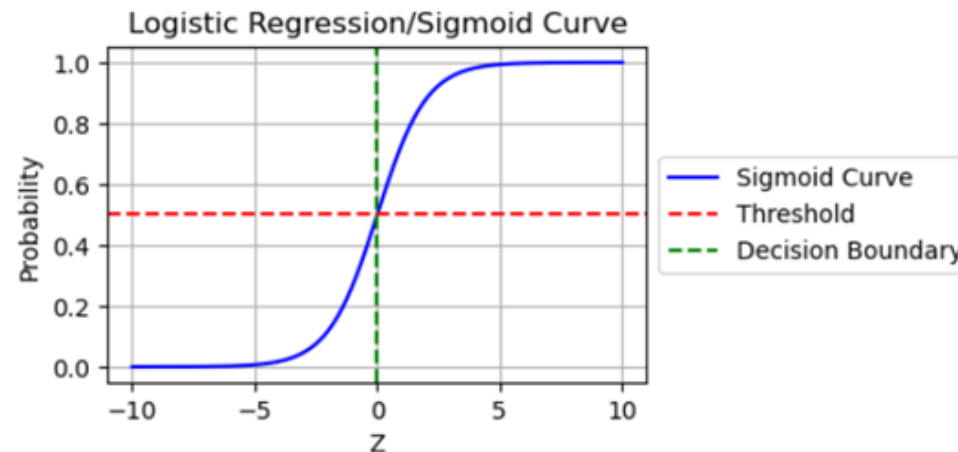
print(f"scikit-learn performance - R-squared: {r2_sk:.4f}, MSE: {mse_sk:.4f}")
```

- Performance: R-squared and Mean Square Errors



Logistic Regression

- a classification model that predicts the probability that the dependent variable (Y) belongs to a binary outcome (0 or 1) based on one or more independent variables (X)
 - achieved by using the logistic function (or sigmoid function) to constrain the output between 0 and 1

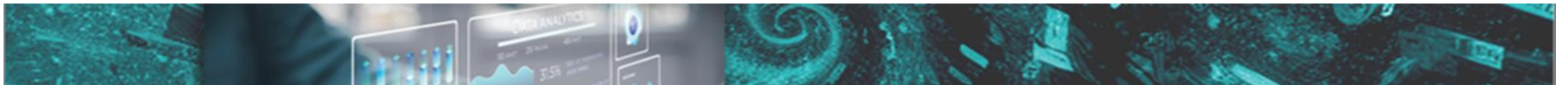


Logistic Regression

- using statsmodels
- achieved by using the logistic function (or sigmoid function) to constrain the output between 0 and 1

```
import statsmodels.api as sm
# Fit the logistic regression model with intercept
logit_model = sm.Logit(y, sm.add_constant(X)) # Adding constant for intercept
result = logit_model.fit()
print(result.summary2())
```

- Performance measure:
 - R-squared value
 - p-values for each independent variable
 - entropy
 - LLR p-value



Logistic Regression

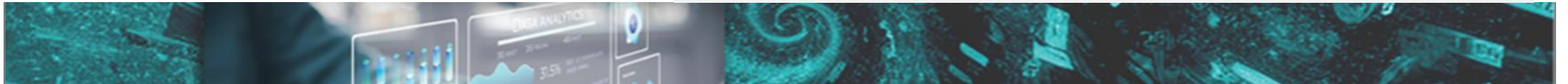
- scikit-learn library
- Performance measure:
 - accuracy_score
 - classification_report

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
# Initialize the logistic regression model
log_reg_sk = LogisticRegression(random_state=0)

# Train the model
log_reg_sk.fit(X_train, y_train)
# Make predictions
y_pred = log_reg_sk.predict(X_test)

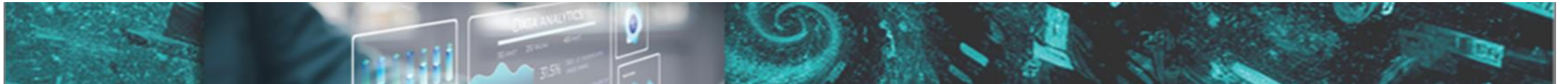
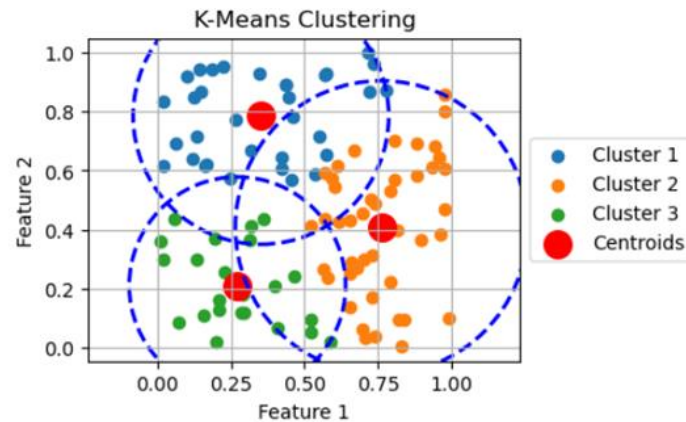
# Evaluate the model
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy}")
from sklearn.metrics import confusion_matrix,
classification_report
# Confusion Matrix
conf_matrix = confusion_matrix(y_test, y_pred)
print("\nConfusion Matrix:")
print(conf_matrix)

# Classification Report (Precision, Recall, F1-Score)
class_report = classification_report(y_test, y_pred)
print("\nClassification Report:")
print(class_report)
```



K-means Clustering

- centers solely on the independent variables (X) without predicting a dependent variable
 - used to group data points into K distinct clusters based on similarities in their features
 - the number of clusters (K) must be predetermined



K-means Clustering

- Two techniques help determine the optimal number of clusters: the elbow method and silhouette analysis.
 - In the elbow method, a plot illustrates the change in Within-Cluster Sum of Squared Errors (WCSS) for different k values. The optimal k value is typically at the "elbow" point in the curve where the WCSS starts to decrease more slowly.
 - Silhouette analysis involves calculating a silhouette coefficient for each data point, ranging from -1 to 1. The average silhouette score across all points measures overall cluster separation, with higher scores indicating better separation.

